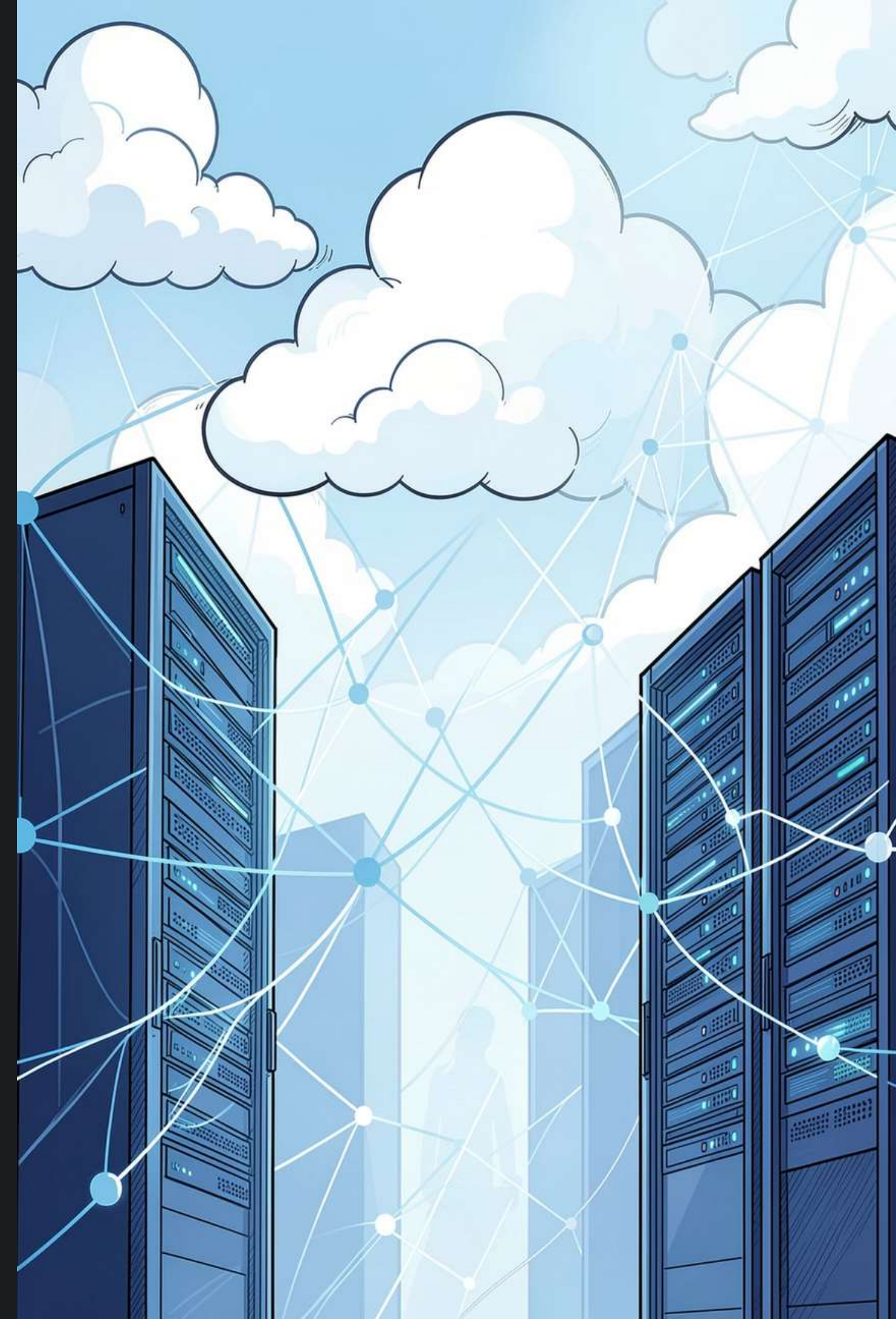
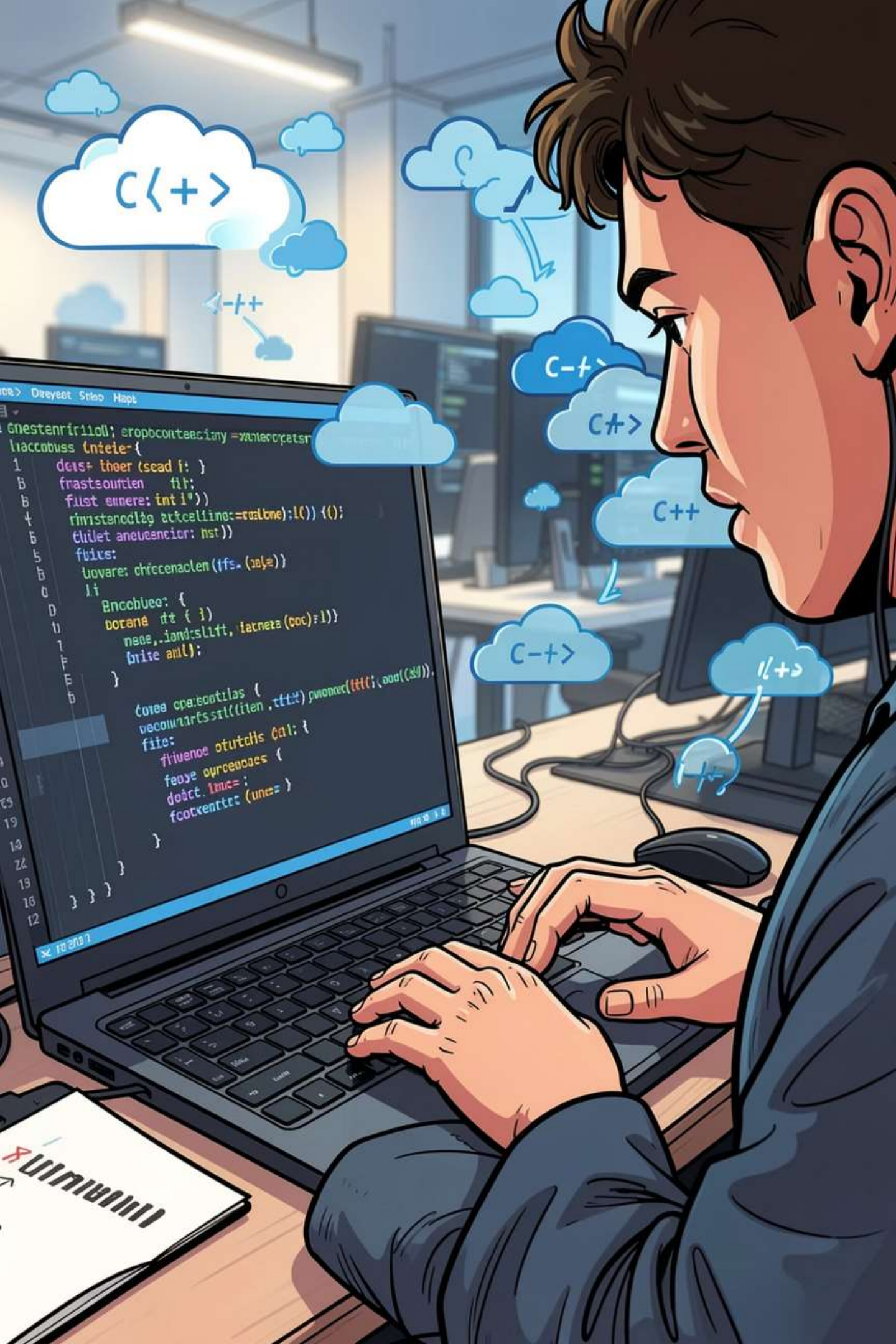


CloudVault Engine

Intelligent Cloud Storage & Deduplication System

Overview: C++-based storage manager optimizing capacity and performance via deduplication, version control, network routing, and load balancing.





What CloudVault Solves

Storage Efficiency

Remove duplicates, compress footprint, lower costs

Data Integrity

Versioning and verification to prevent and recover corruption

Network & Scale

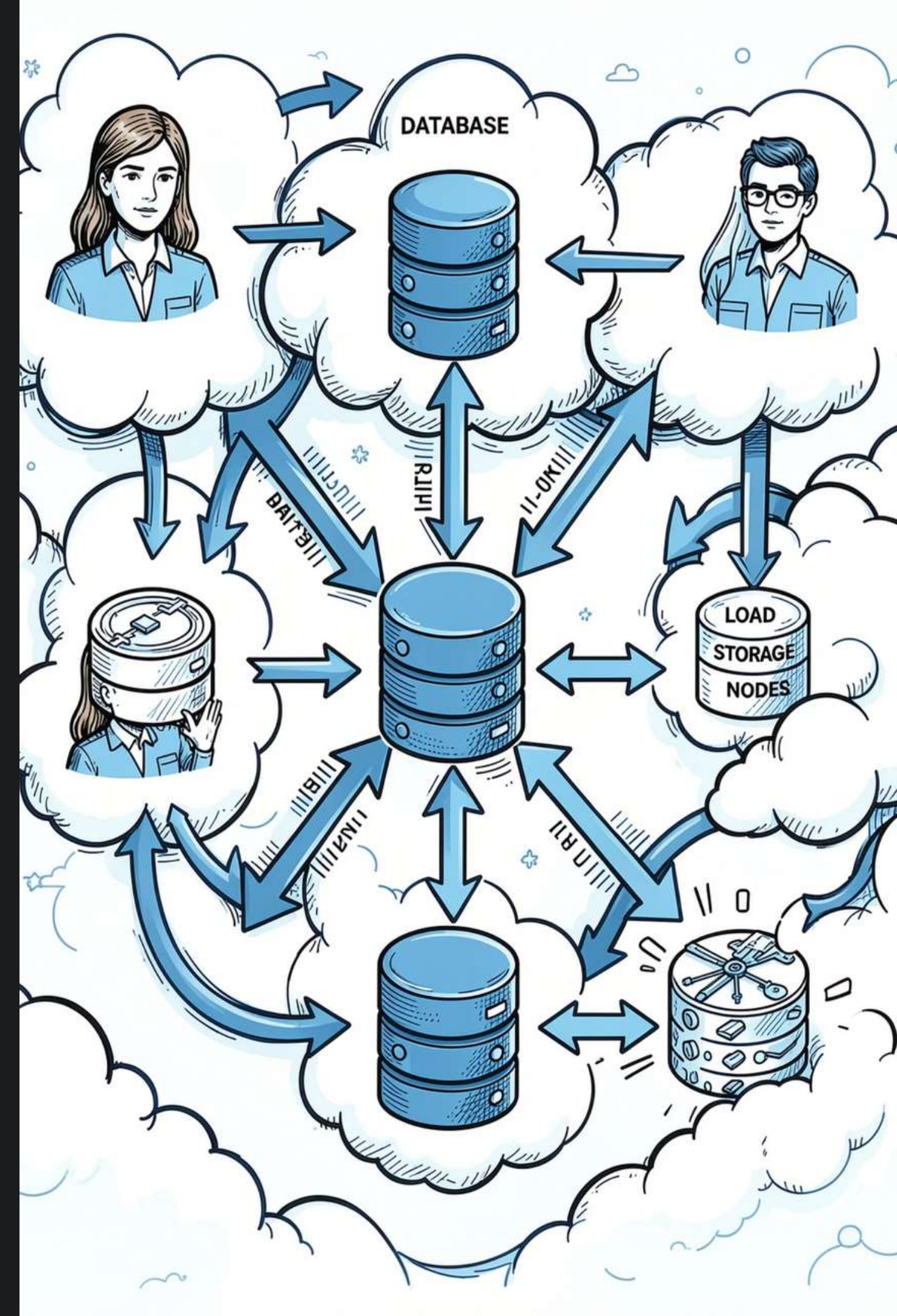
Smart routing and load distribution for consistent performance

System Architecture

High-level flow (left → right):

1. User Input
2. Folder Directory (Trie)
3. Disk Version Manager
4. Deduplication Engine
5. Verification Queue
6. Storage Optimizer
7. Storage Network
8. Load Balancer

Icons: cloud storage, database, verification, optimizer, load balancer — visualized above for quick mapping.



Architecture Flowchart (Process)



```
graph LR; A[User Input] --> B[Trie Directory]; B --> C[Disk Versioning]; C --> D[Deduplication];
```

User Input

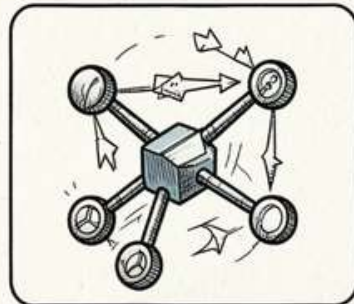
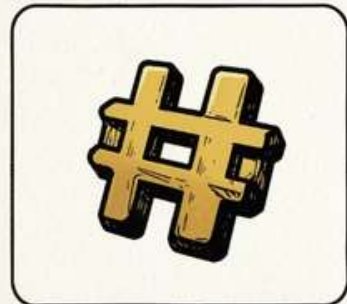
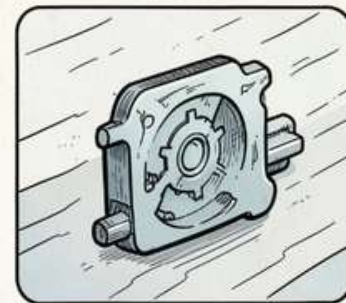
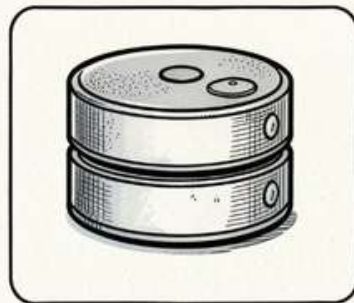
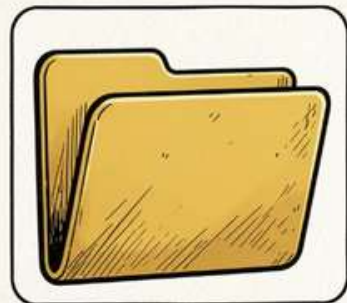
Trie Directory

Disk Versioning

Deduplication

Each stage represents a pipeline step with clear handoff and specialized responsibilities for throughput and correctness.

Core Modules



Folder Directory (Trie)

Fast path lookup, memory-efficient nodes, autocomplete support for UI and APIs.



Verification Queue

FIFO processing, fair scheduling, background integrity checks.



Disk Version Manager

Stores multiple revisions, atomic commits, rollback on corruption detection.



Deduplication Engine

FNV-1a hashing pipeline, chunking strategy, content-addressable storage for duplicates.

Algorithms & Data Structures



Folder Search

Trie — $O(k)$ lookup, prefix autocomplete



Version Control

Hash map + vector for compact version lists and fast access



Verification Queue

FIFO queue with fair scheduling



Deduplication

FNV-1a hashing, chunk fingerprinting, content-addressable index



Optimization

Priority queue (max heap) for scheduling compaction and rebalancing



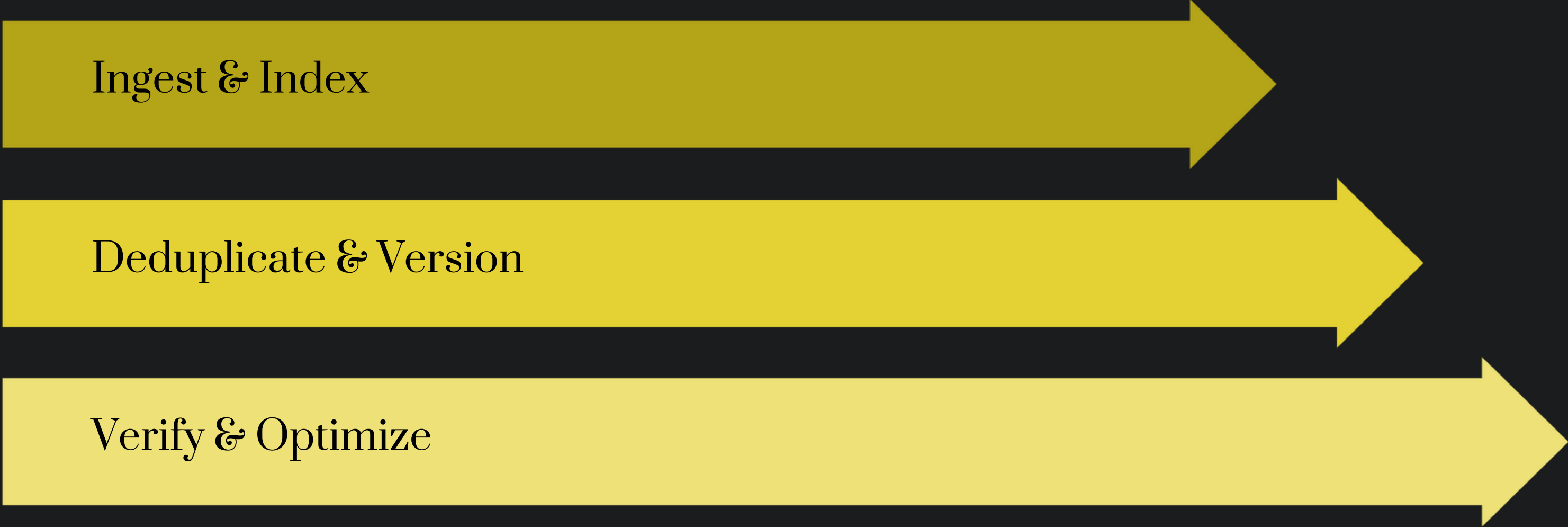
Routing & Load

Dijkstra for path selection; min-heap for server load balancing

Module → Algorithm Mapping

Module	Data Structure / Algorithm
Folder Search	Trie
Version Control	Hash Map + Vector
Verification Queue	FIFO Queue, Fair Scheduler
Deduplication	FNV-1a Hashing, Chunk Index
Optimization	Priority Queue (Max Heap)
Routing	Dijkstra Algorithm
Load Balancing	Min Heap for server load

Processing & Data Flow — Sequence



Ingest & Index

Deduplicate & Version

Verify & Optimize

Pipeline highlights: parallelizable dedupe, async verification, prioritized optimization to minimize I/O spikes.

Results

Efficient Duplicate Detection

Content-addressable index with FNV-1a reduces redundant blocks.

Reduced Storage Usage

Significant footprint savings via chunk-level deduplication and compaction.

Fast Folder Search

Trie-backed lookup and autocomplete for sub-second responses.

Optimal Network Paths & Balanced Load

Dijkstra routing and heap-based workload distribution maintain throughput.

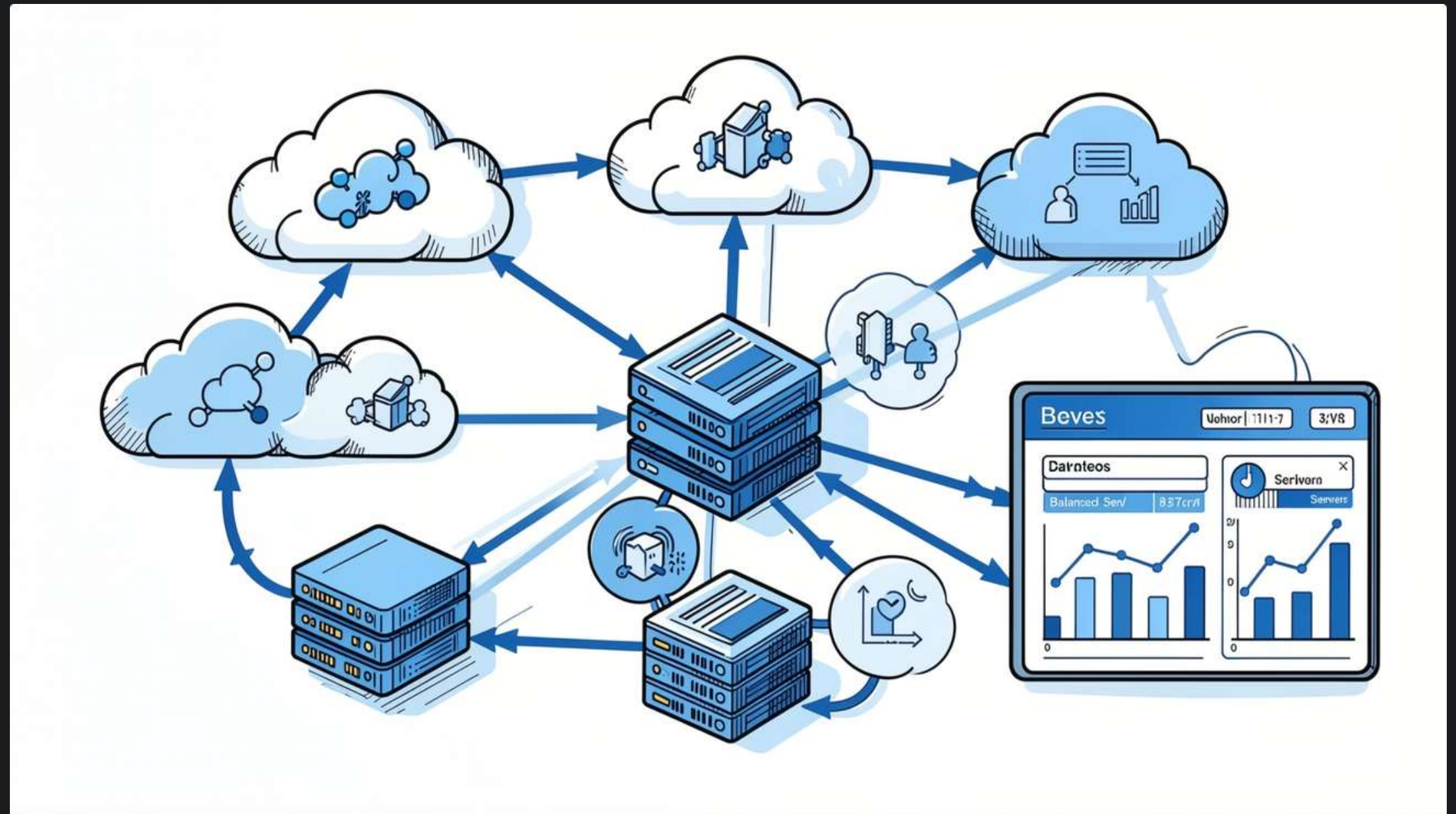
Conclusion & Next Steps

Conclusion

- CloudVault Engine integrates proven DS & algorithms to deliver efficient, reliable cloud storage.
- Design favors asynchronous pipelines, compact indexes, and algorithmic routing for scale.

Future Scope

- Distributed multi-region storage and replication
- Encryption-at-rest and secure dedupe protocols
- Web dashboard and telemetry for ops
- AI-driven storage prediction and tiering



Thank you — Questions & Discussion